

Computer System Design & Application

计算机系统设计与应用

陶伊达 (TAO Yida)

taoyd@sustech.edu.cn

Course Logistics

- Course website: Blackboard <https://bb.sustech.edu.cn/>
- [Slides and other resources will all be uploaded here.](#)
- Office hours: Wednesday
10:00 – 12:00 pm, CoE
South Building, 411B

Lecturer: Yida Tao (陶伊达), taoyd@sustech.edu.cn.

Lab tutor: Yao Zhao (赵耀), zhaoy6@sustech.edu.cn

理论课

周一34节, 商学院103

实验课

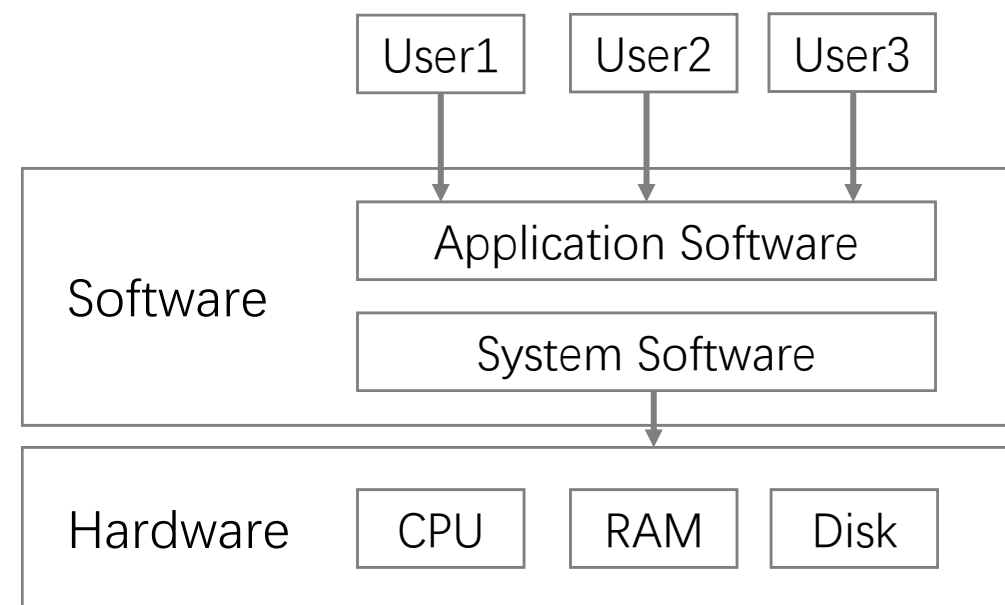
1组: 周一56节, 三教501。SA: 崔彦儒 12432662@mail.sustech.edu.cn

2组: 周三34节, 三教507。SA: 张海涵 12432718@mail.sustech.edu.cn

3组: 周三56节, 三教507。SA: 刘锦润 12432683@mail.sustech.edu.cn

Computer System

- Hardware
 - The physical parts: CPU, keyboard, disks
- Software
 - System software: a set of **programs** that control & manage the operations of hardware, e.g., OS
 - Application software: a set of **programs** for end users to perform specific tasks, e.g., browser, media player



Our focus?

Topics covered

Foundations

- OOP, Functional programming
- Software design concepts
- JVM architecture

.....

Utilities

- Exception handling
- Generic collections
- Lambdas & Streams
- Annotation
- Reflection

.....

Functionalities

- File I/O
- Networking
- Multithreading
- Web development
- GUI

.....

Applications

- Data analytics & visualization
- C/S applications
- Web development
- Web services
- Testing

.....

Syllabus

(Negotiable)

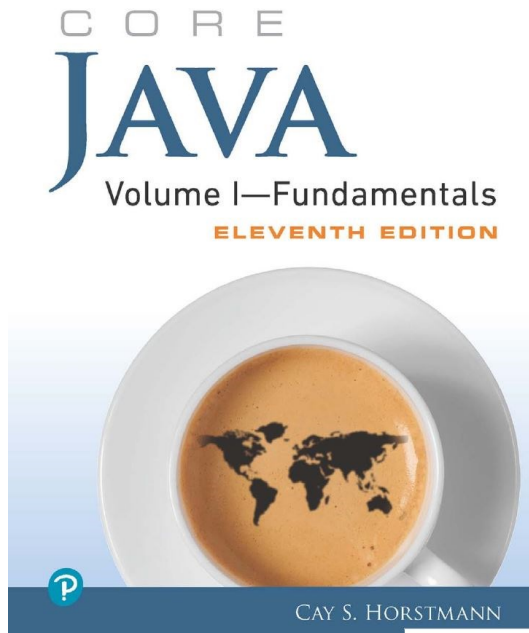
- Lecture 1: Computing overview, OOP, Design Principles
- Lecture 2: Generics, ADT, Collections
- Lecture 3: Functional programming, Lambda
- Lecture 4: Java 8 Stream API
- Lecture 5: I/O Streams, Encoding
- Lecture 6: Serialization, File I/O, Exception Handling
- Lecture 7: Concurrency, Multithreading
- Lecture 8: Network Programming
- Lecture 9: Reflection, Annotation
- Lecture 10: GUI Intro, JavaFX
- Lecture 11: Java EE, Servlet
- Lecture 12: The Spring Framework
- Lecture 13: Spring Boot
- Lecture 14: Logging, JUnit Testing
- Lecture 15: JVM

Coursework & Grading Policy

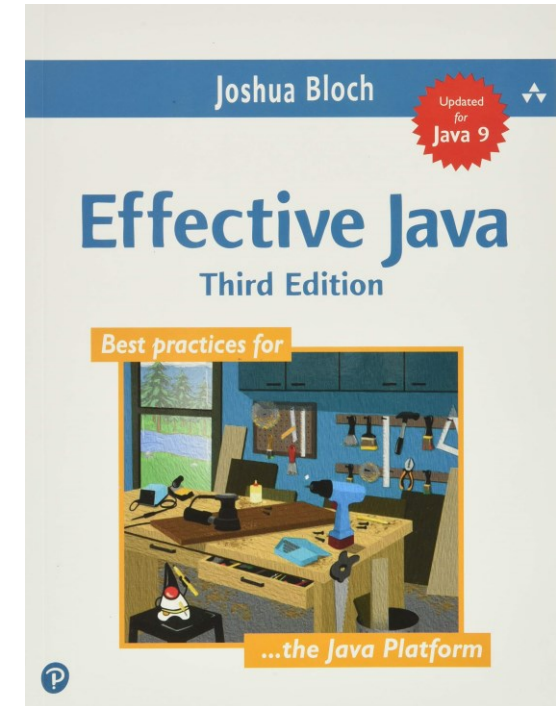
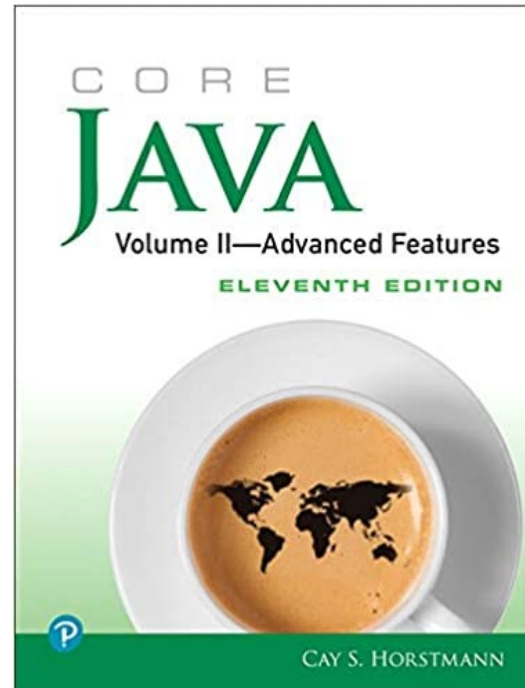
	Score	Description
Assignment 1	10%	Release at week 4 and due at week 7
Assignment 2	15%	Release at week 8 and due at week 11 +1 (max) for presenting at week 16 lecture
Project	20%	Released at around week 10 Team: 2 people +0.5 for submitting the final project at week 15 +1 (max) for presenting at week 16 lecture
Labs	15%	Attendance Lab practices (+0.1 for finishing onsite. max +1)
Quiz	10%	Quizzes, exercises, participation during lectures
Final Exam	30%	Close-book (Two pieces of A4 cheat sheets allowed) No electronic device

Labs start from the 1st week!

Reference Books



Core Java Volume I II
Cay S. Horstmann



Effective Java
Joshua Bloch

An abstract graphic on the left side of the slide, featuring concentric circles and various digital patterns like binary code and pixelated shapes in shades of blue, green, and white.

Lecture 1

- Course introduction
- Software design principles
- OO concepts
- Java and JVM

Software Design Principles

- High Cohesion, Low Coupling (高内聚、低耦合)
- Information Hiding (信息隐藏)
- Code Reuse (代码复用)

Cohesion & Coupling

- Modules (模块): A complex software system can be divided into simpler pieces called *modules*
- Cohesion (内聚): How elements of a module are functionally related to each other
- Coupling (耦合): How different modules depend on each other



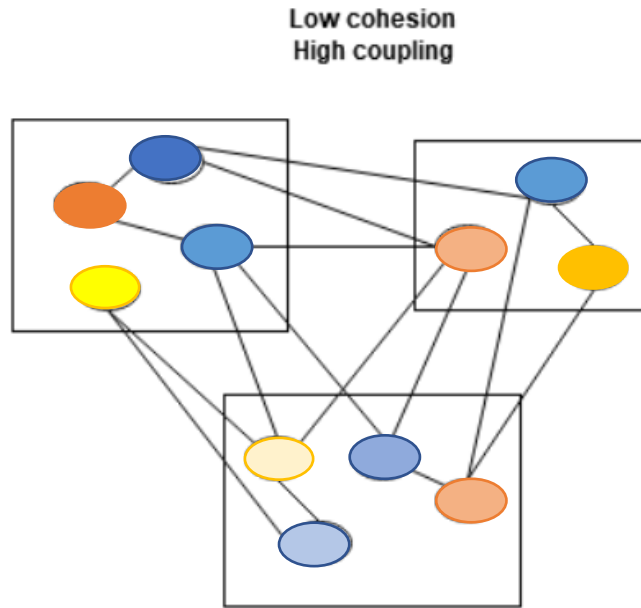
Cohesion & Coupling

- **High cohesion:** modules are self-contained and have a single, well-defined purpose; all of its elements are directly related to the functionality that is meant to be provided by the module
- **Low coupling:** modules should be as independent as possible from other modules, so that changes to one module will have minimal impact on other modules

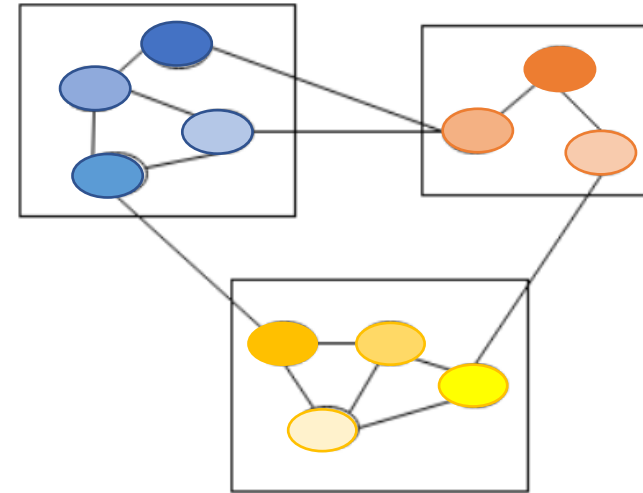


High Cohesion, Low Coupling

Difficult to read,
understand, reuse,
test, and maintain



High cohesion
Low coupling



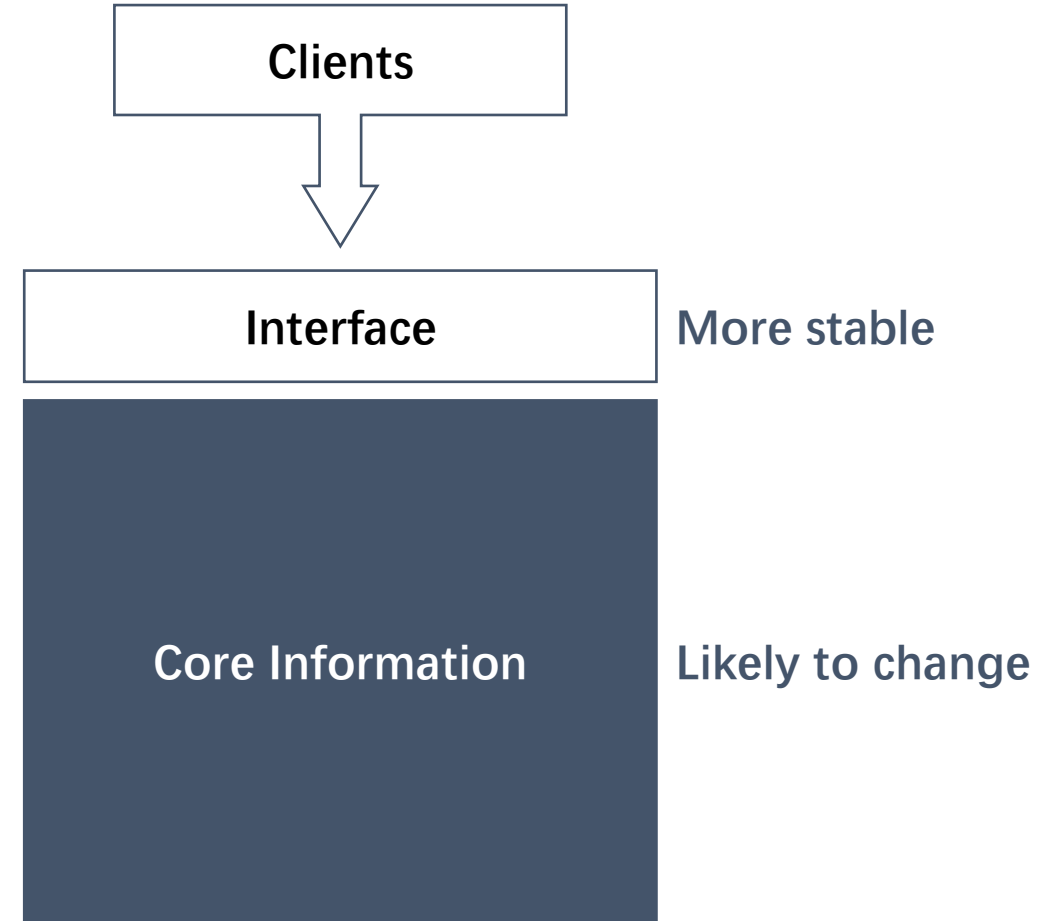
Easy to understand,
extend, and modify

Software Design Principles

- High Cohesion, Low Coupling (高内聚、低耦合)
- Information Hiding (信息隐藏)
- Code Reuse (代码复用)

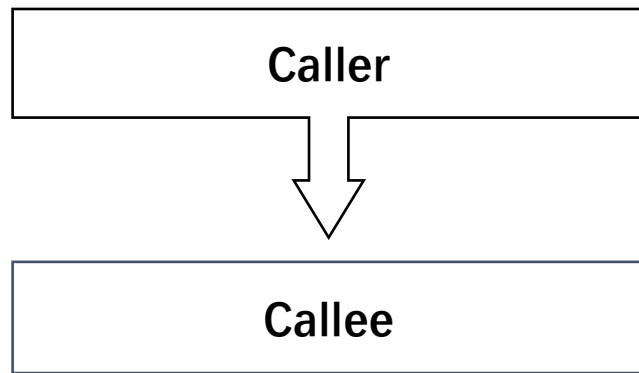
Information Hiding

- Hiding certain information, such as design decisions, data, and implementation details, from client programs; only expose the necessary
- Pros: Client programs won't have to change even if the core design or implementation is changed



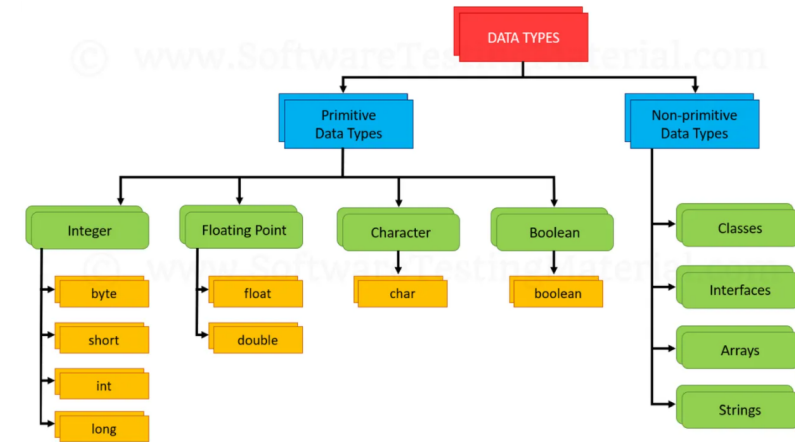
Information Hiding

Example 1. Function Call



The caller function doesn't have to know how the callee function works internally; it only has to know callee's arguments and return type

Example 2. Data Representation



You don't need to know how a data type is implemented in order to use it;

Software Design Principles

- High Cohesion, Low Coupling (高内聚、低耦合)
- Information Hiding (信息隐藏)
- Code Reuse (代码复用)

Code Reuse

- Reusable code refers to the use of the same source code collection in multiple applications or systems.
- At its best, code reuse is accomplished by sharing common classes or collections of functions and procedures. At its worst, code reuse is accomplished by copying and then modifying existing code.
- In OOP, code reusability is to design objects in a way that can later on be used on other systems.

OO Concepts

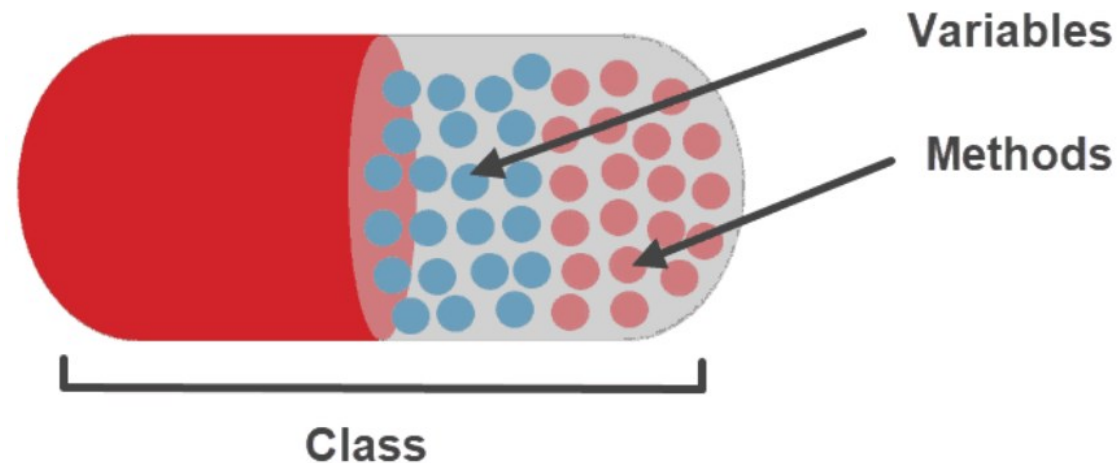
- Encapsulation (封装)
- Inheritance (继承)
- Abstraction (抽象)
- Polymorphism (多态)

Software Design Principles

- High Cohesion, Low Coupling (高内聚、低耦合)
- Information Hiding (信息隐藏)
- Code Reuse (代码复用)

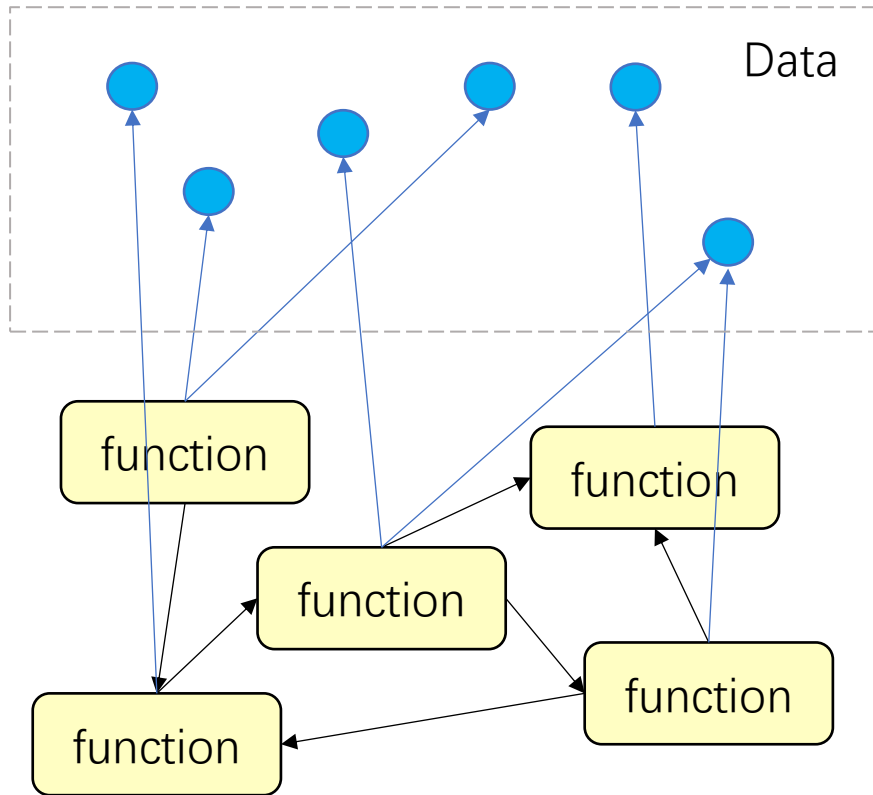
Encapsulation

- Bundling the data and behaviors which operate on that data into a single unit, e.g., a class in Java.
- Program should interact with object data *only* through the object's methods.



How do we achieve encapsulation in Java?

Procedural Design

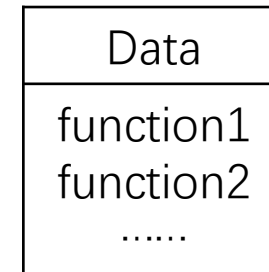
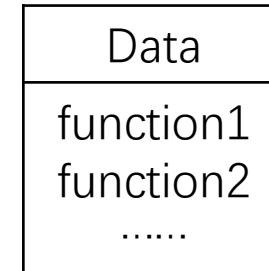
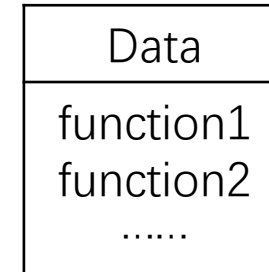


**High coupling. Bad information hiding.
Hard to make changes and to scale.**

Object-oriented Design



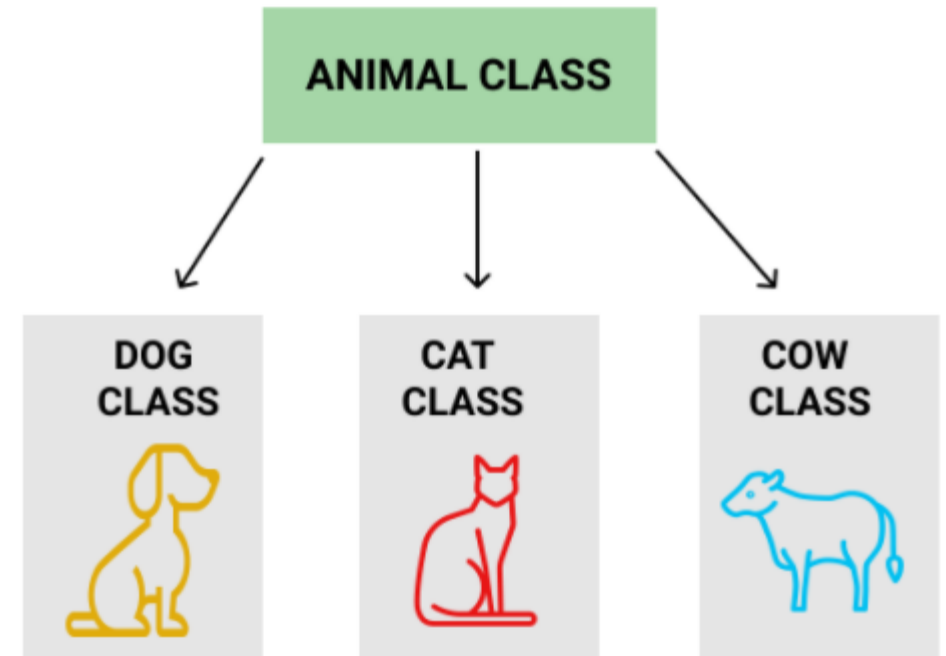
Traffic Control System



**High cohesion. Good information hiding.
Easier to maintain and extend.**

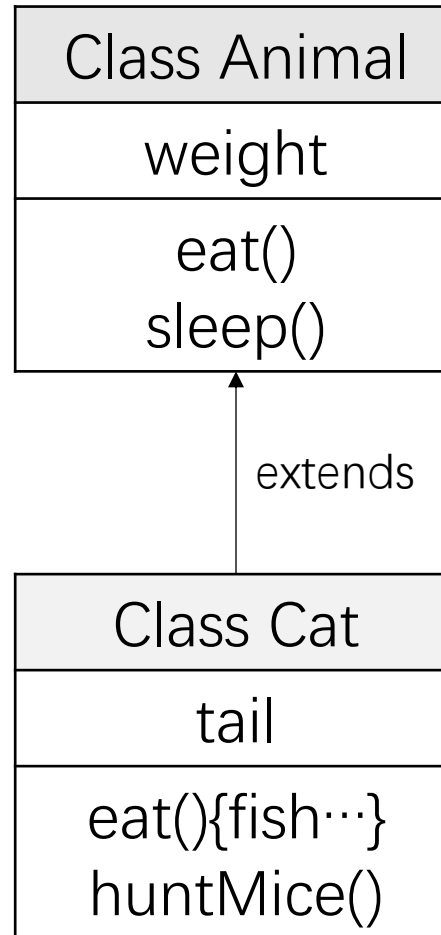
Inheritance

- Motivation: objects are similar and share common logics
- Inheritance allows a new class (subclass) to be created by deriving variables and methods from an existing class (superclass)



Inheritance

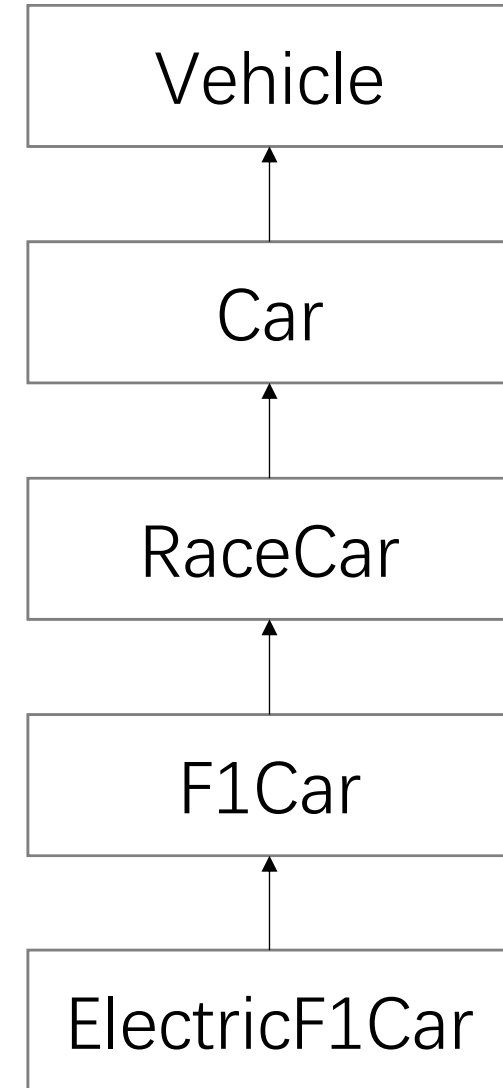
- Subclass could **reuse** inherited field directly (**weight**)
- Subclass could declare new fields (**tail**)



- Subclass could **reuse** inherited method directly (**sleep()**)
- Subclass could override methods in superclass (**eat()**)
- Subclass could declare new methods (**huntMice()**)

Inheritance

- Inheritance encourages code reuse
- Too much inheritance = high coupling
- Balance inheritance with other tools, e.g., interfaces or composition

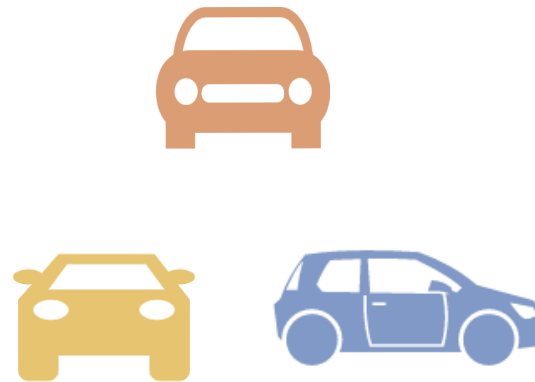


Abstraction

- Achieved in Java by **interface** and **abstract class**
- Abstraction solves problems at design level, by simplifying complex systems by **exposing only the necessary details**.

Abstract **Car** Class

Color Size Model
start() stop() move() turn()

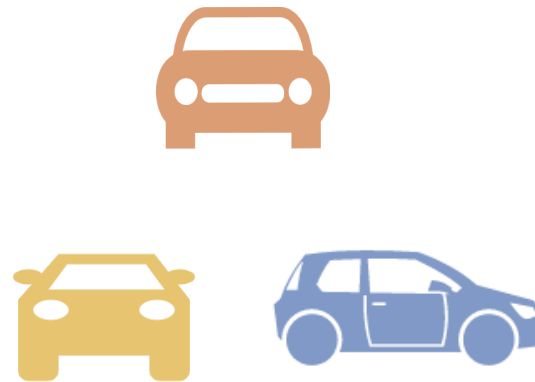


Abstraction

- **High cohesion:** the class has one clear responsibility and nothing more

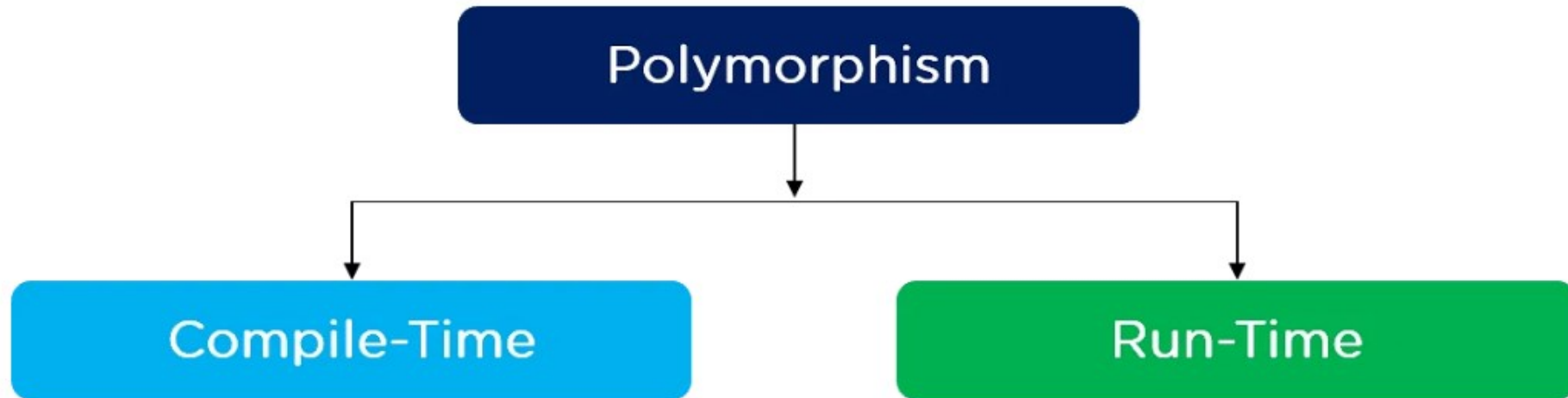
Abstract **Car** Class

Color Size Model
start() stop() move() turn()

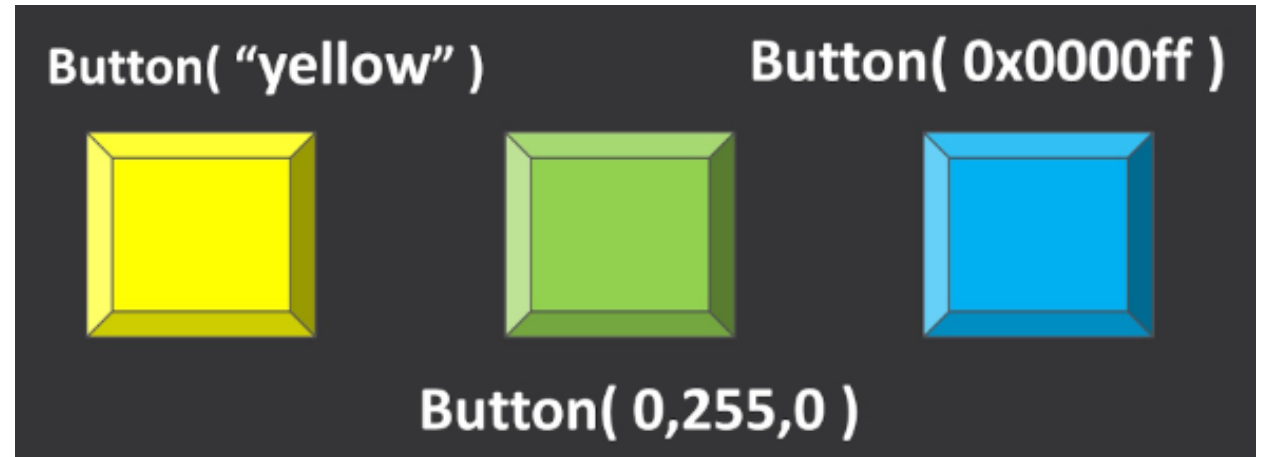
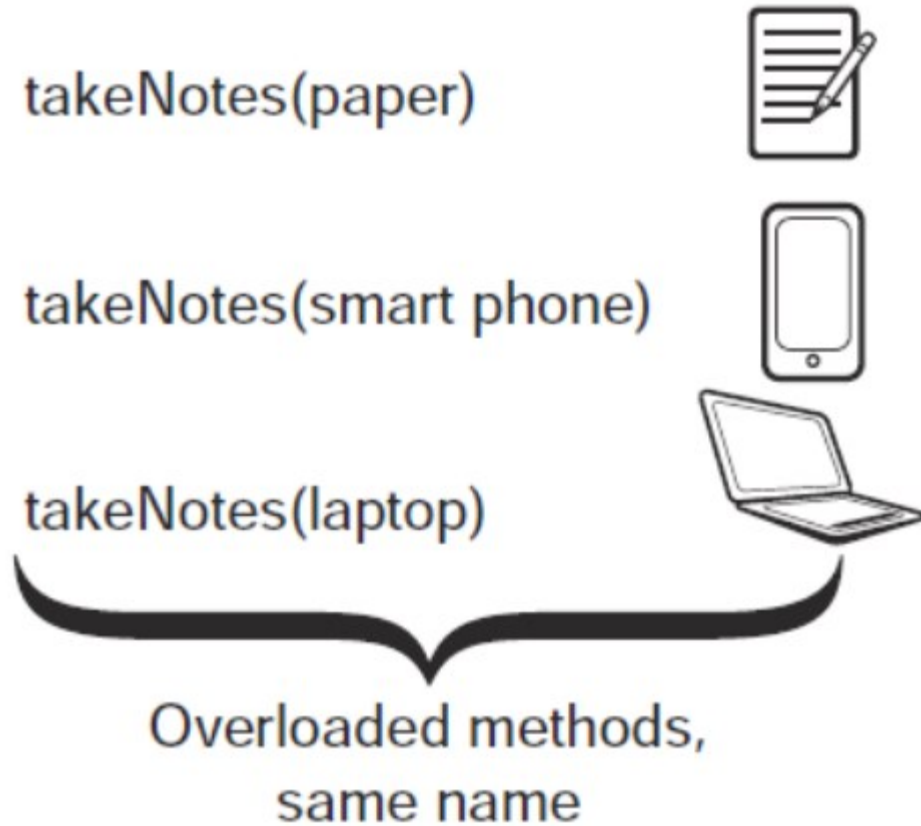


Polymorphism

In general, "polymorphism" refers to the ability of a single entity or concept to take on multiple forms or have multiple meanings.

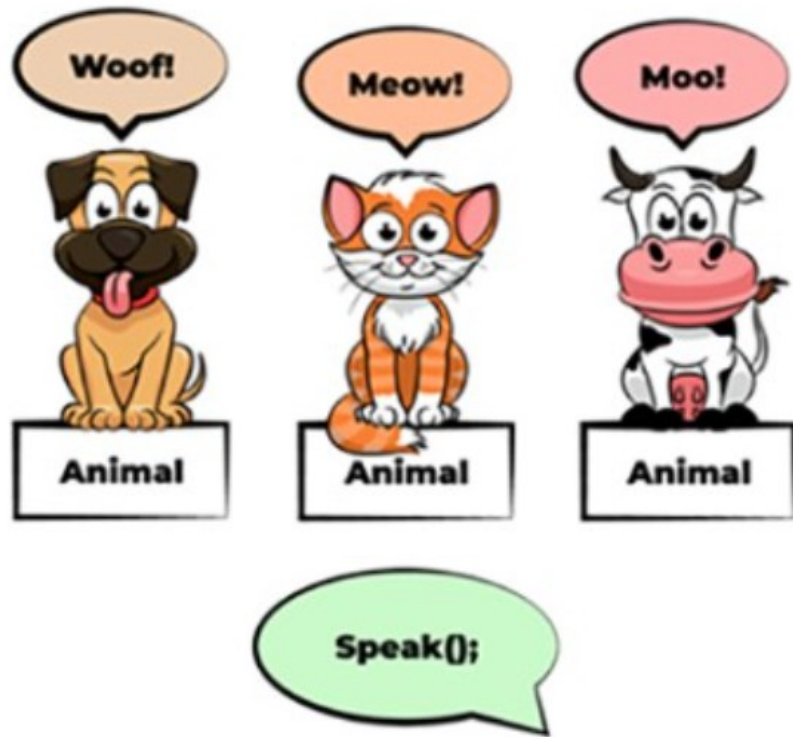


Compile-time Polymorphism

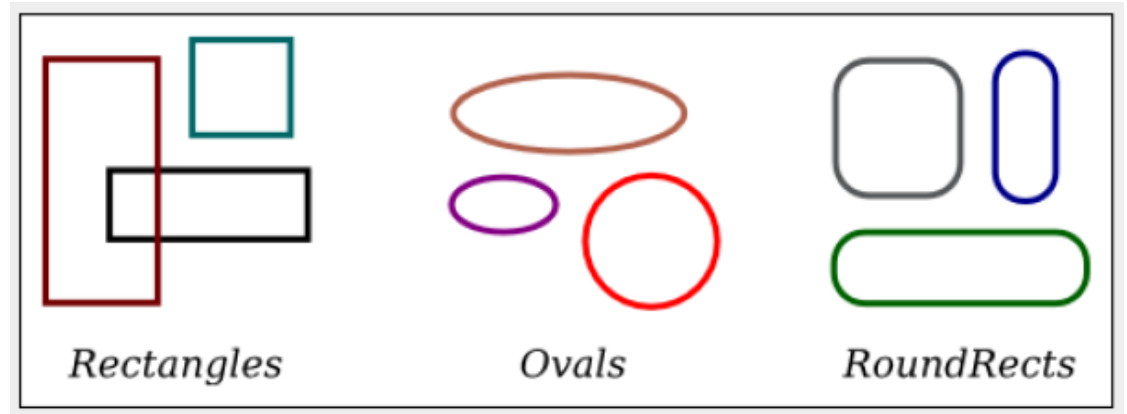


Method overloading are resolved using static binding,
in which mapping is resolved at compile time

Runtime Polymorphism



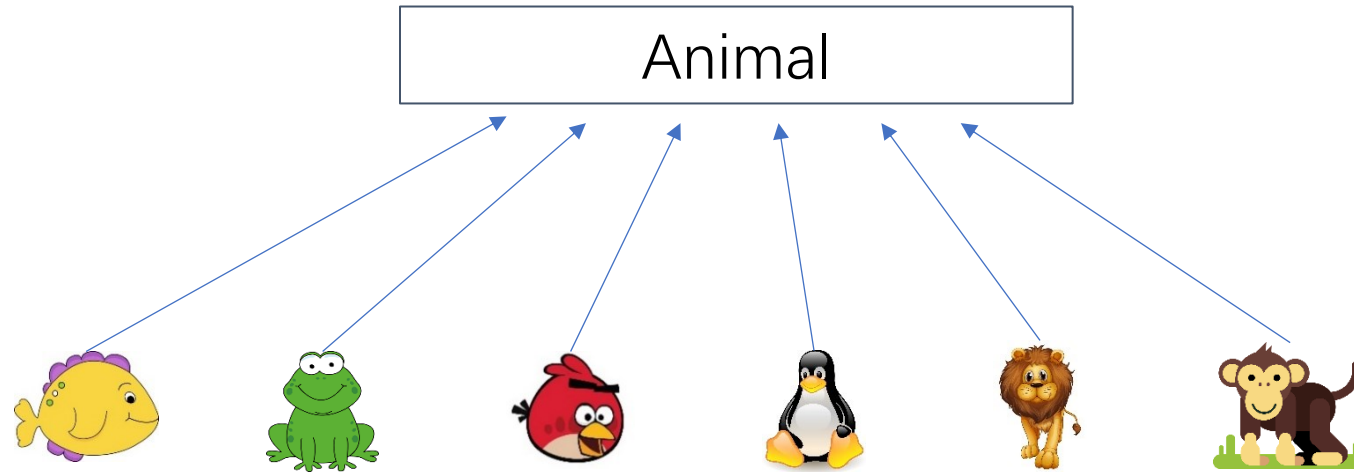
```
for (int i = 0; i < shapelist.length; i++ ) {  
    Shape shape = shapelist[i];  
    shape.redraw();  
}
```



Method overriding are resolved using dynamic binding, in which the mapping is resolved at execution time

Polymorphism

- **Low coupling:** we can add Horse or Snake later without changing the code that already uses Animal



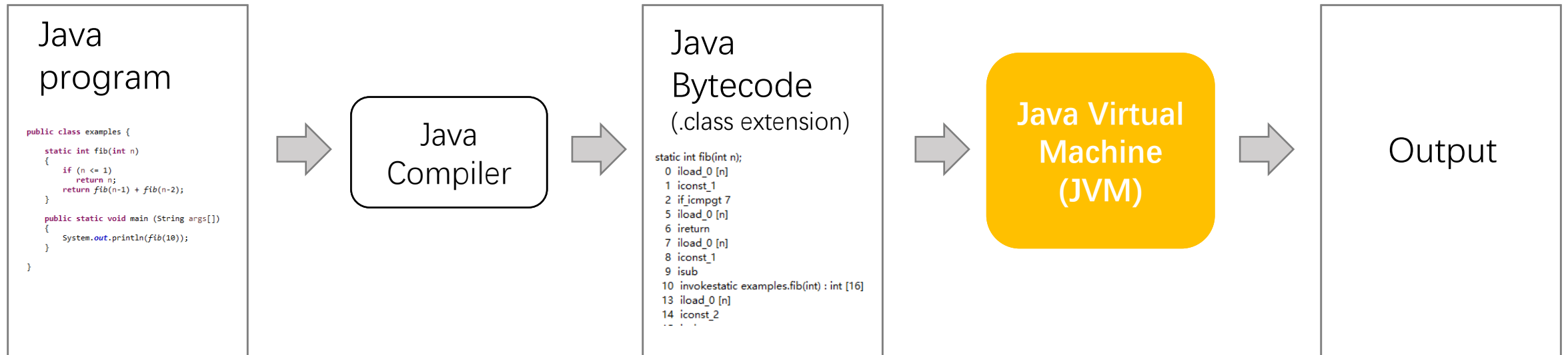


Lecture 1

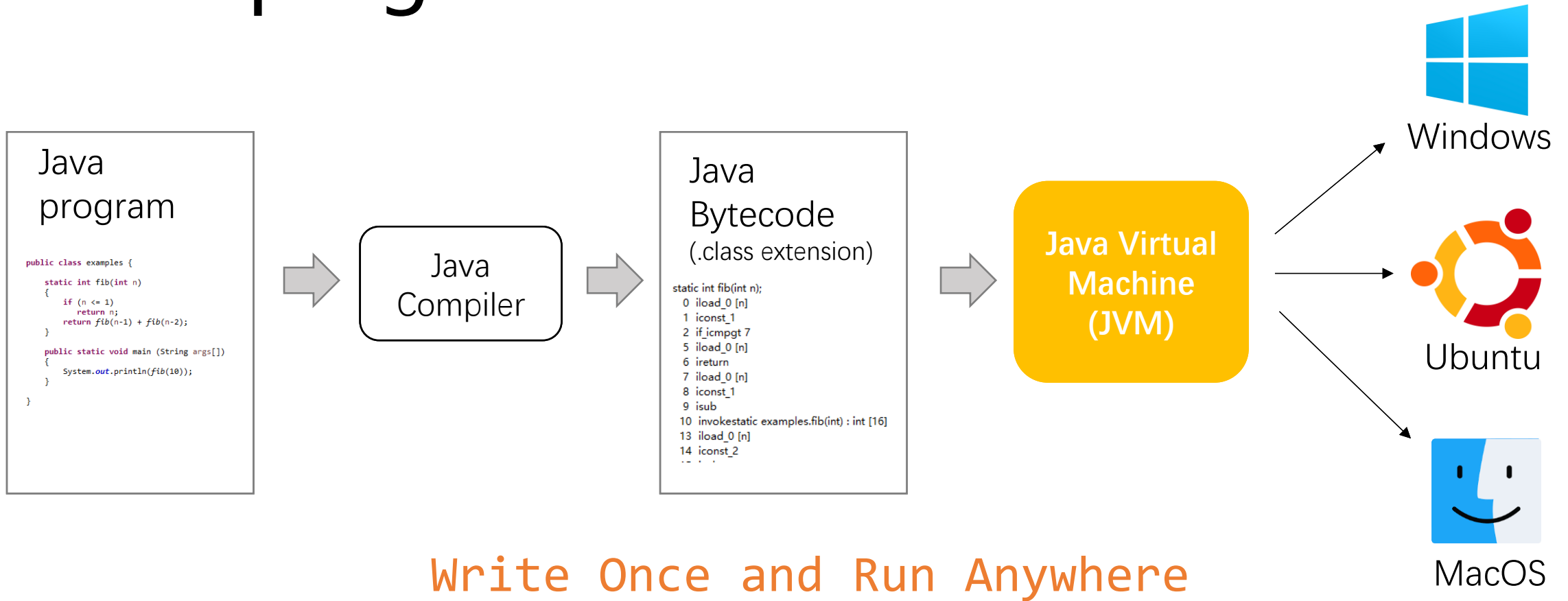
- Course introduction
- Software design principles
- OO concepts
- Java and JVM

Java program execution

- High-level source → low-level/machine code



Java program execution

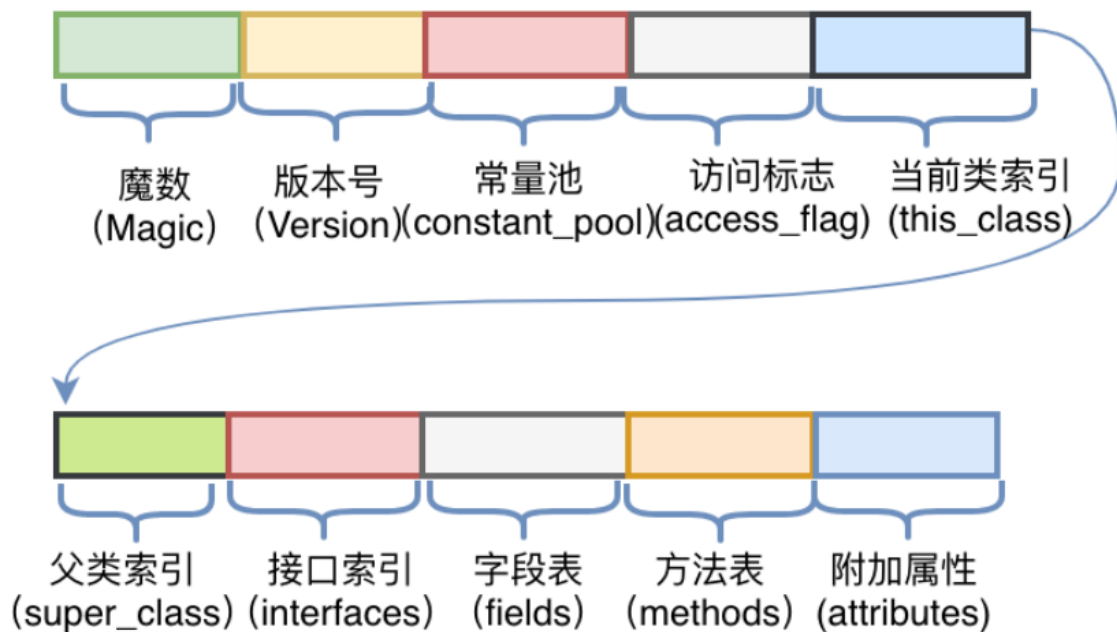


.class: Platform Independence

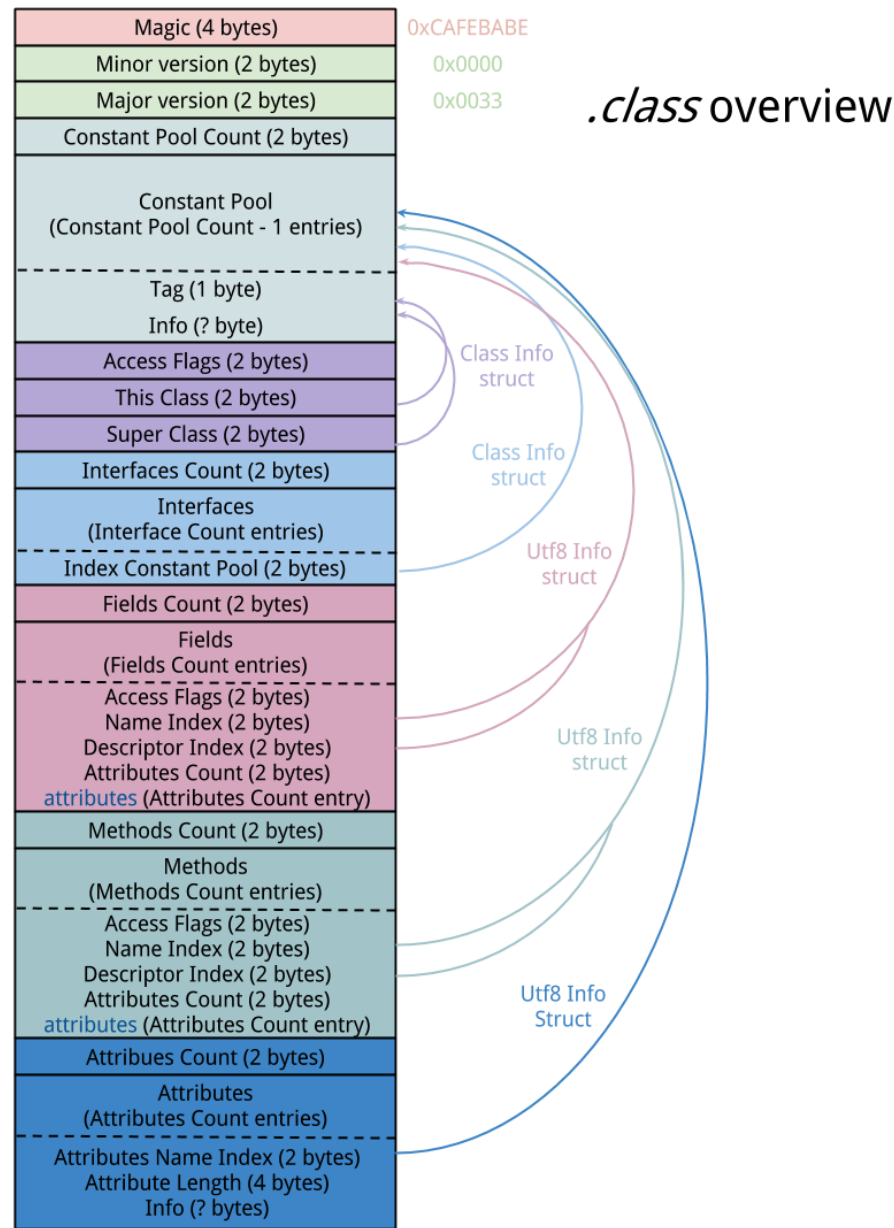


Image source: 《深入理解Java虚拟机》第三版, 周志明

.class file format



<https://github.com/likuisuper/Java-Notes/>



<https://blog.lse.epita.fr/2014/04/28/0xcafebabe-java-class-file-format-an-overview.html>

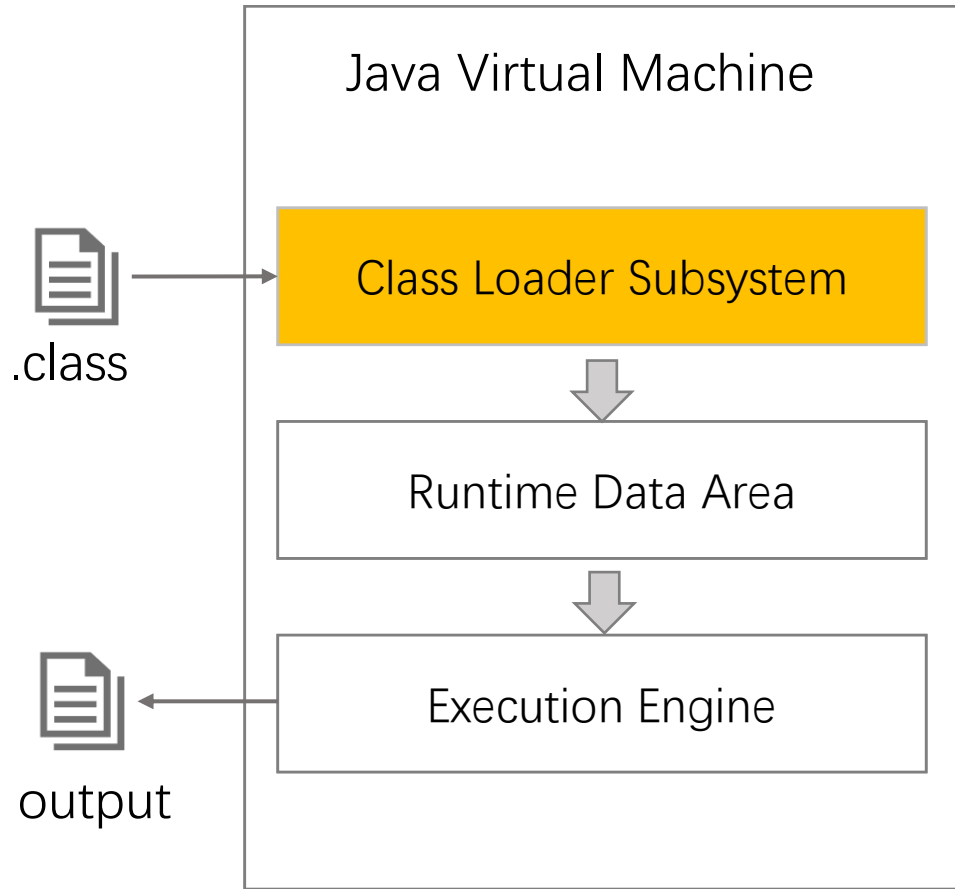
Version Number

Java is backward compatible: you can run Java 15 compiled jar on JRE 16 but not vice versa.

`java.lang.UnsupportedClassVersionError (version 60.0)` happens because of a higher JDK (JDK 16) during compile time and lower JDK during runtime.

Java SE	Major Version
18	62
17	61
16	60
15	59
14	58
13	57
12	56
11	55
10	54
9	53
8	52
7	51
6.0	50
5.0	49
1.4	48
1.3	47
1.2	46
1.1	45

Java Virtual Machine (JVM)



Class Loader

- Locating and loading necessary .class or .jar (Java **AR**chive, aggregations of .class files) files into memory
 - .jar that offers standard Java packages (e.g., java.lang, java.io)
 - .class and .jar (dependency) for your application, which is specified in *classpath*
- Errors occur when class loader fails to locate a required .class

ClassLoader: Loading

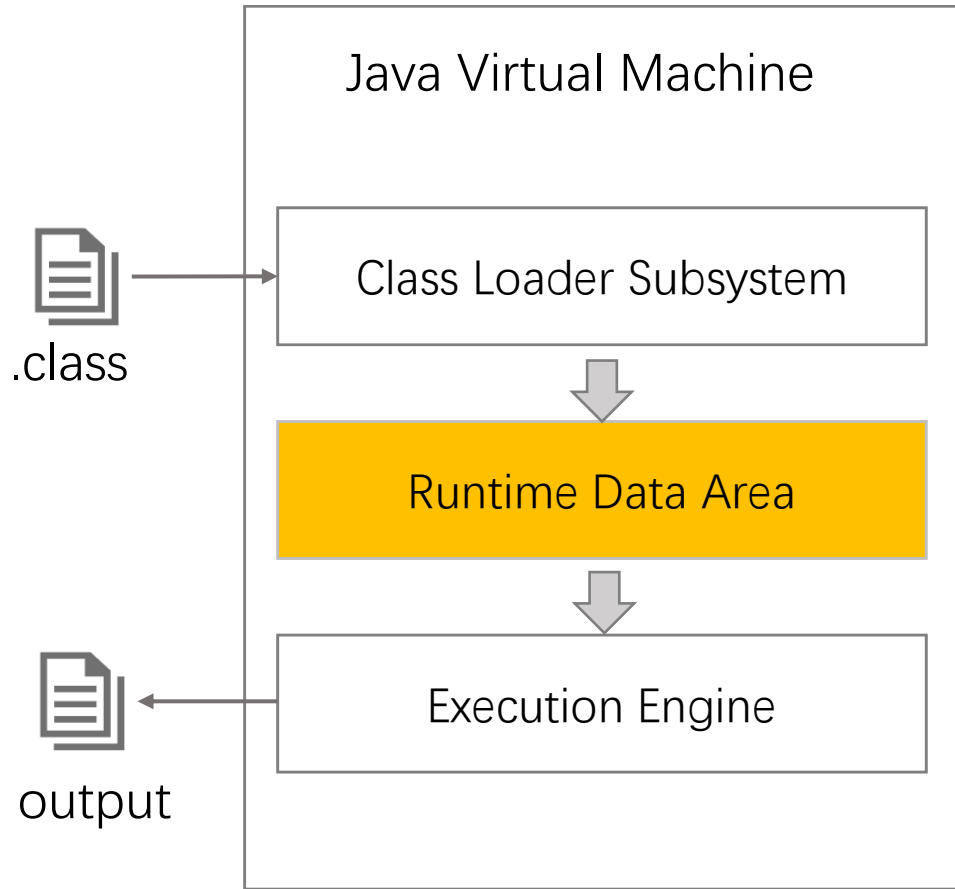
```
public class HelloApp {  
    public static void main(String argv[]) {  
        System.out.println("Aloha! Hello and Bye");  
    }  
}
```

Image source: <https://stackoverflow.com/questions/2424604/what-is-a-java-classloader>

```
prmp>java -verbose:cl class HelloApp
```

```
[Opened C:\Program Files\Java\jre1.5.0\lib\rt.jar]  
[Opened C:\Program Files\Java\jre1.5.0\lib\jsse.jar]  
[Opened C:\Program Files\Java\jre1.5.0\lib\jce.jar]  
[Opened C:\Program Files\Java\jre1.5.0\lib\charsets.jar]  
[Loaded java.lang.Object from shared objects file]  
[Loaded java.io.Serializable from shared objects file]  
[Loaded java.lang.Comparable from shared objects file]  
[Loaded java.lang.CharSequence from shared objects file]  
[Loaded java.lang.String from shared objects file]  
[Loaded java.lang.reflect.GenericDeclaration from shared objects file]  
[Loaded java.lang.reflect.Type from shared objects file]  
[Loaded java.lang.reflect.AnnotatedElement from shared objects file]  
[Loaded java.lang.Class from shared objects file]  
[Loaded java.lang.Cloneable from shared objects file]  
[Loaded java.lang.ClassLoader from shared objects file]  
[Loaded java.lang.System from shared objects file]  
[Loaded java.lang.Throwable from shared objects file]  
.  
.  
.  
[Loaded java.security.BasicPermissionCollection from shared objects file]  
[Loaded java.security.Principal from shared objects file]  
[Loaded java.security.cert.Certificate from shared objects file]  
[Loaded HelloApp from file:/C:/classes/]  
Aloha! Hello and Bye  
[Loaded java.lang.Shutdown from shared objects file]  
[Loaded java.lang.Shutdown$Lock from shared objects file]
```

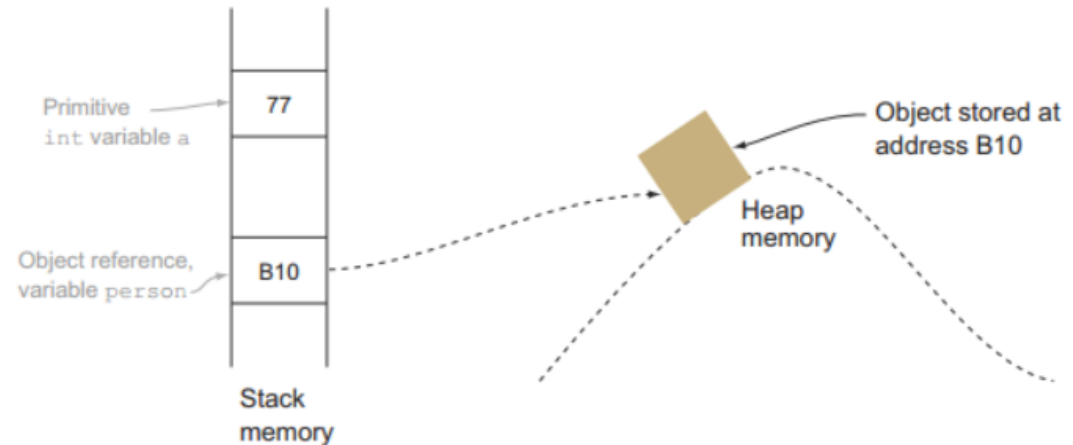
Java Virtual Machine (JVM)



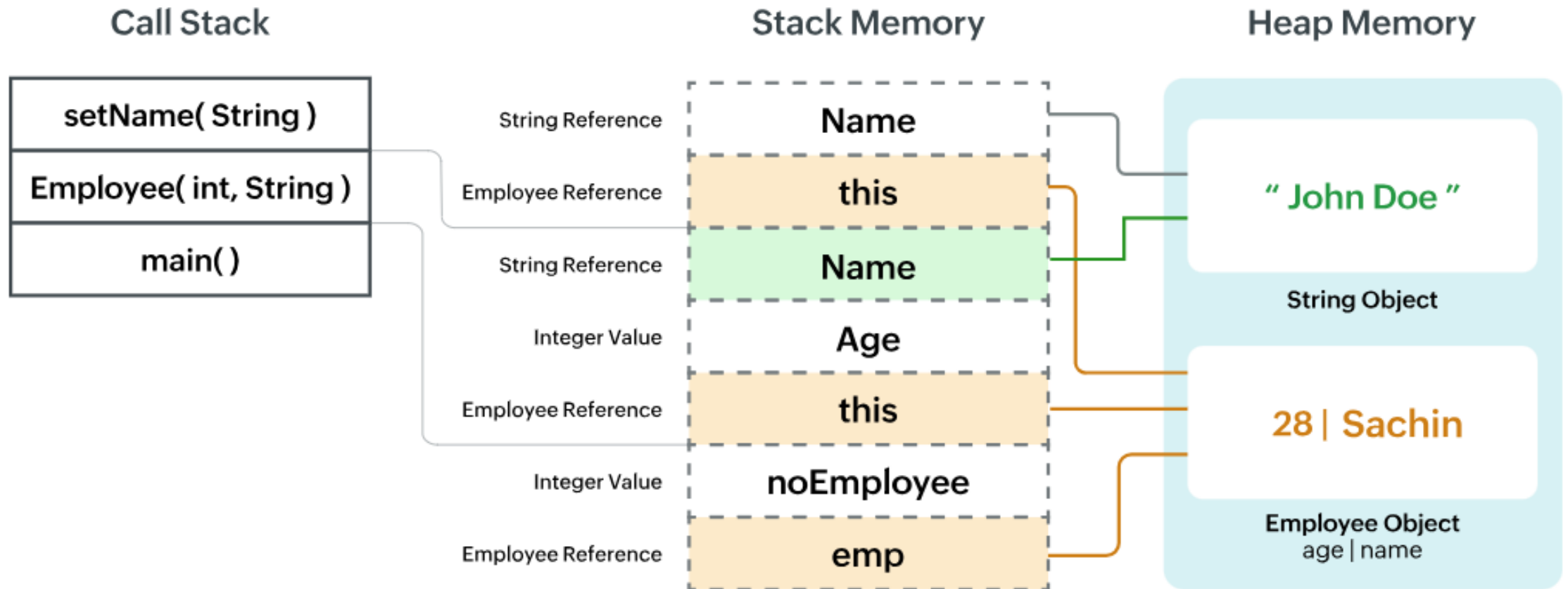
Runtime Data Area

Store all kinds of data and information

- Class-level data in **Method Area**
- Objects/instances in **Heap Area**
- Local variables in **Stack Area**

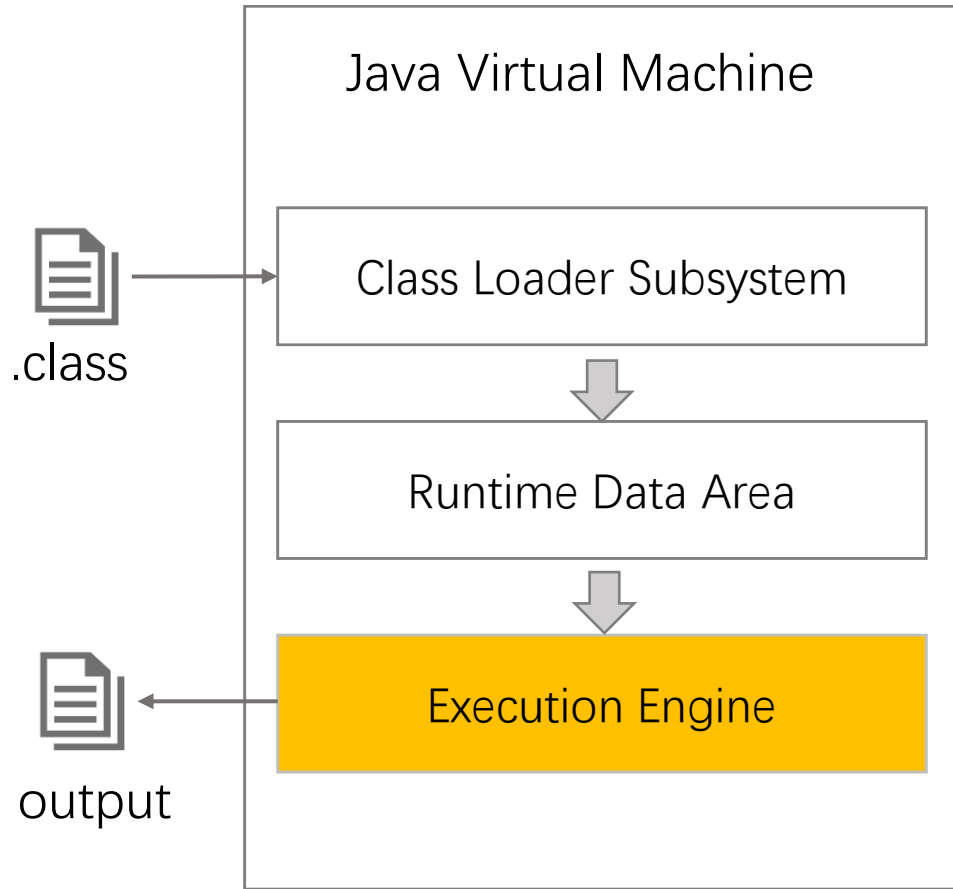


Memory Allocation



<https://www.site24x7.com/learn/java/heap-and-stack-memory-management.html>

Java Virtual Machine (JVM)

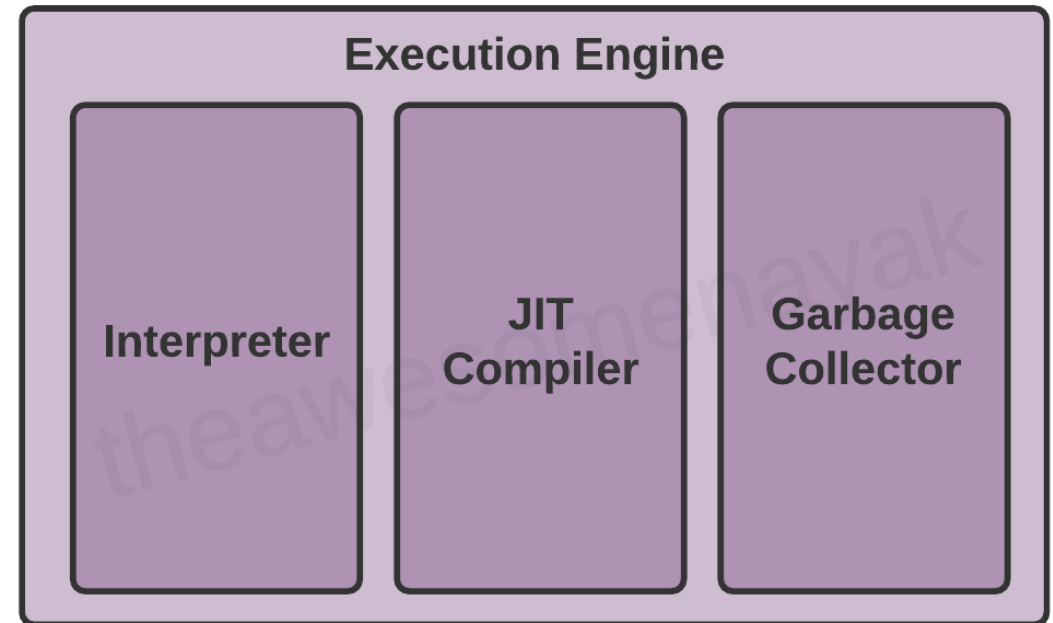


Execution Engine

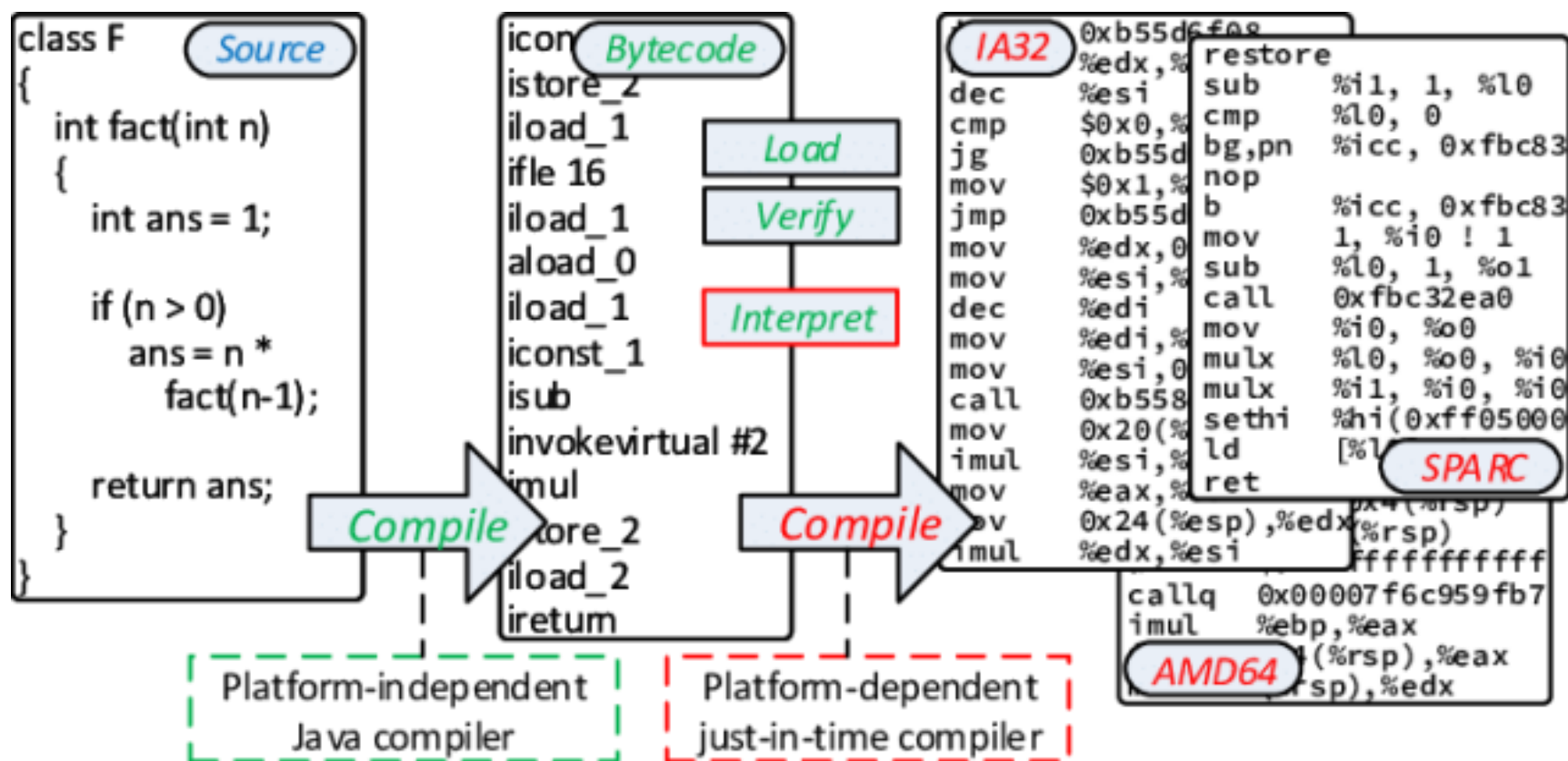
- CPU instructions are platform-dependent: Your computer's processor (like Intel, AMD, or ARM) understands machine code, not Java bytecode.
- Execution engine: translating "run anywhere" .class code to "run on this particular machine" instructions

Execution Engine: Interpreter & JIT Compiler

- Mainstream JVMs initially use Interpreter (解释器) to interpret and execute bytecode
- Yet, this is inefficient for Hot Spot code:
 - Methods that are invoked many times
 - Loops that are executed frequently
- For better efficiency, JIT compiler (即时编译器) compiles and optimize Hot Spot code to local machine code

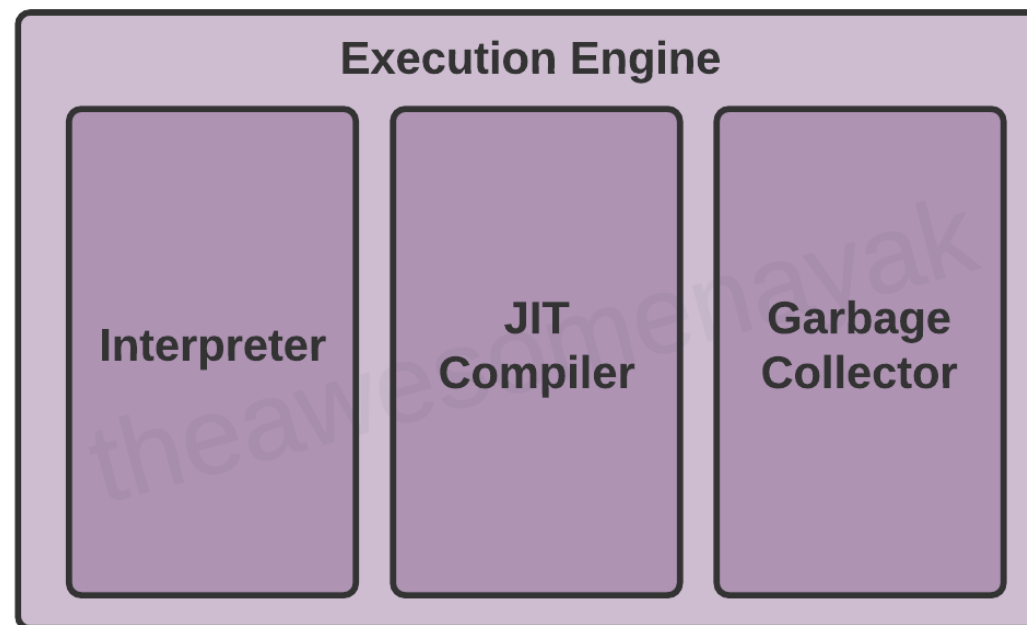


Execution Engine: The Process



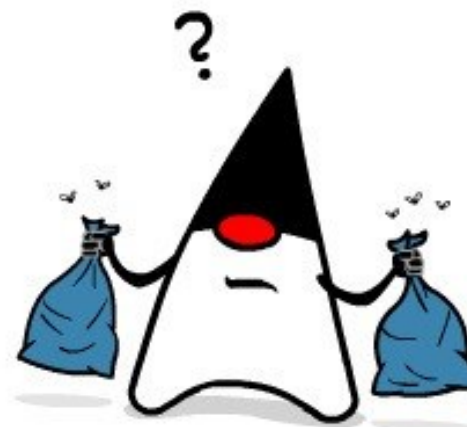
Execution Engine: Garbage Collection

- **Stack area:** static memory allocation; memory can be computed once the class information is loaded
- **Heap area:** dynamic memory allocation; memory is known only during runtime (how many objects to create or to destroy is unknown until we execute the program)
- Garbage collection focuses on the heap area



Which Objects are “Garbage” ?

- 引用计数算法
(Reference Counting)
- 可达性分析算法
(Reachability Analysis)



OverOps

Reference Counting

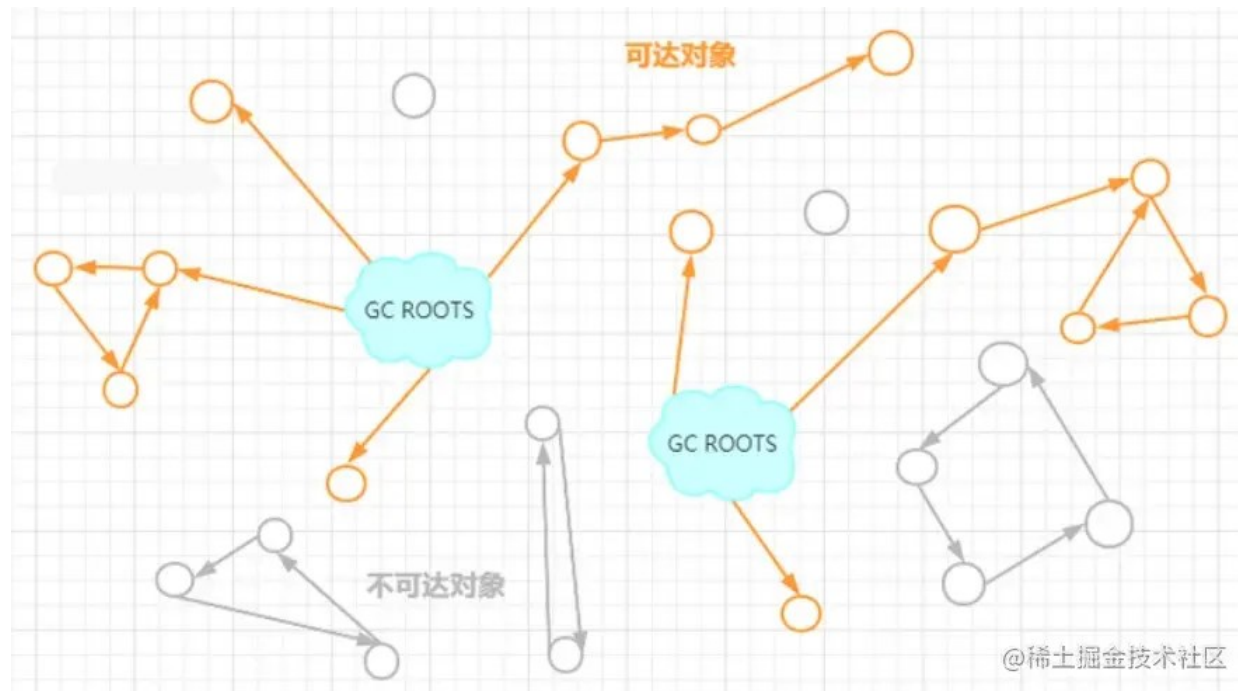
Reference counting GC algorithms associate a reference count with each object.

These algorithms consider an object to be alive as long as the number of references to that object is greater than zero.

What to do for **Cyclic References** ?

Reachability Analysis

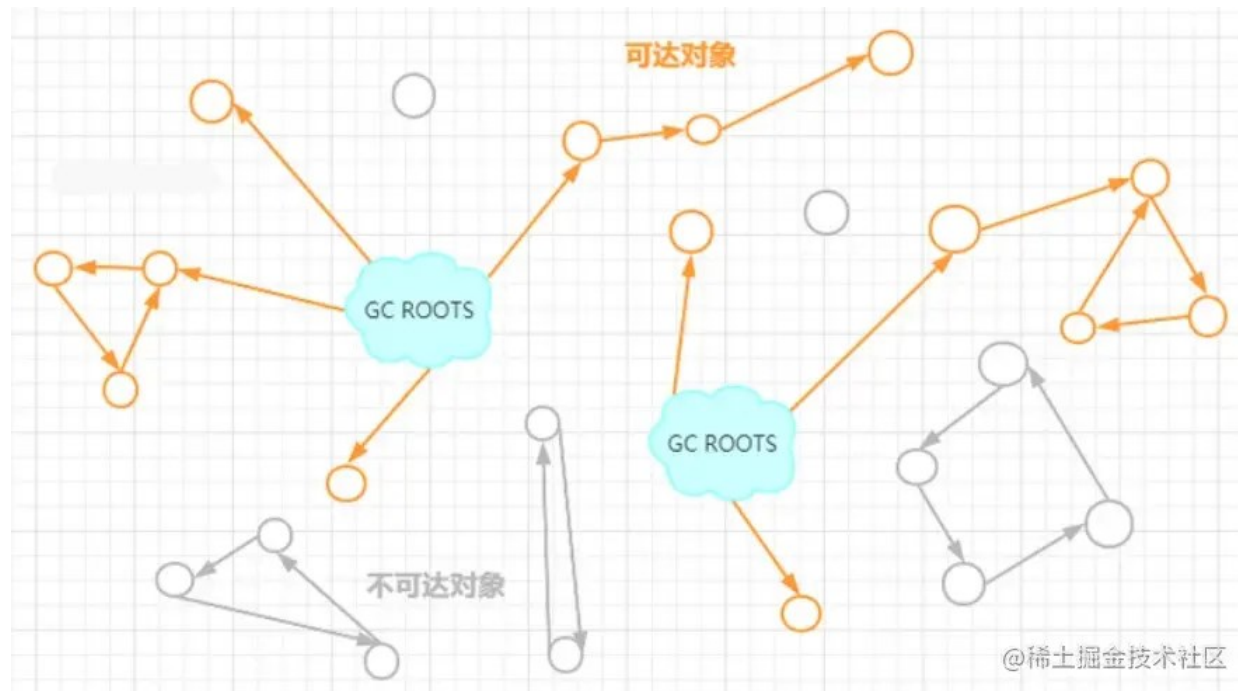
- Starting from GC Roots, find reachable objects iteratively
- Other objects, even with >0 reference count, will be garbage collected



<https://juejin.cn/post/7123853933801373733>

Reachability Analysis

- GC roots are objects that are themselves referenced by the JVM:
 - Local variables in active methods
 - Static variables of loaded class
 -



<https://juejin.cn/post/7123853933801373733>

Next Lecture

- Generics
- Collections